



Django

...

Ingeniería de Software

M.TI. Juan Luis Campos Barrera

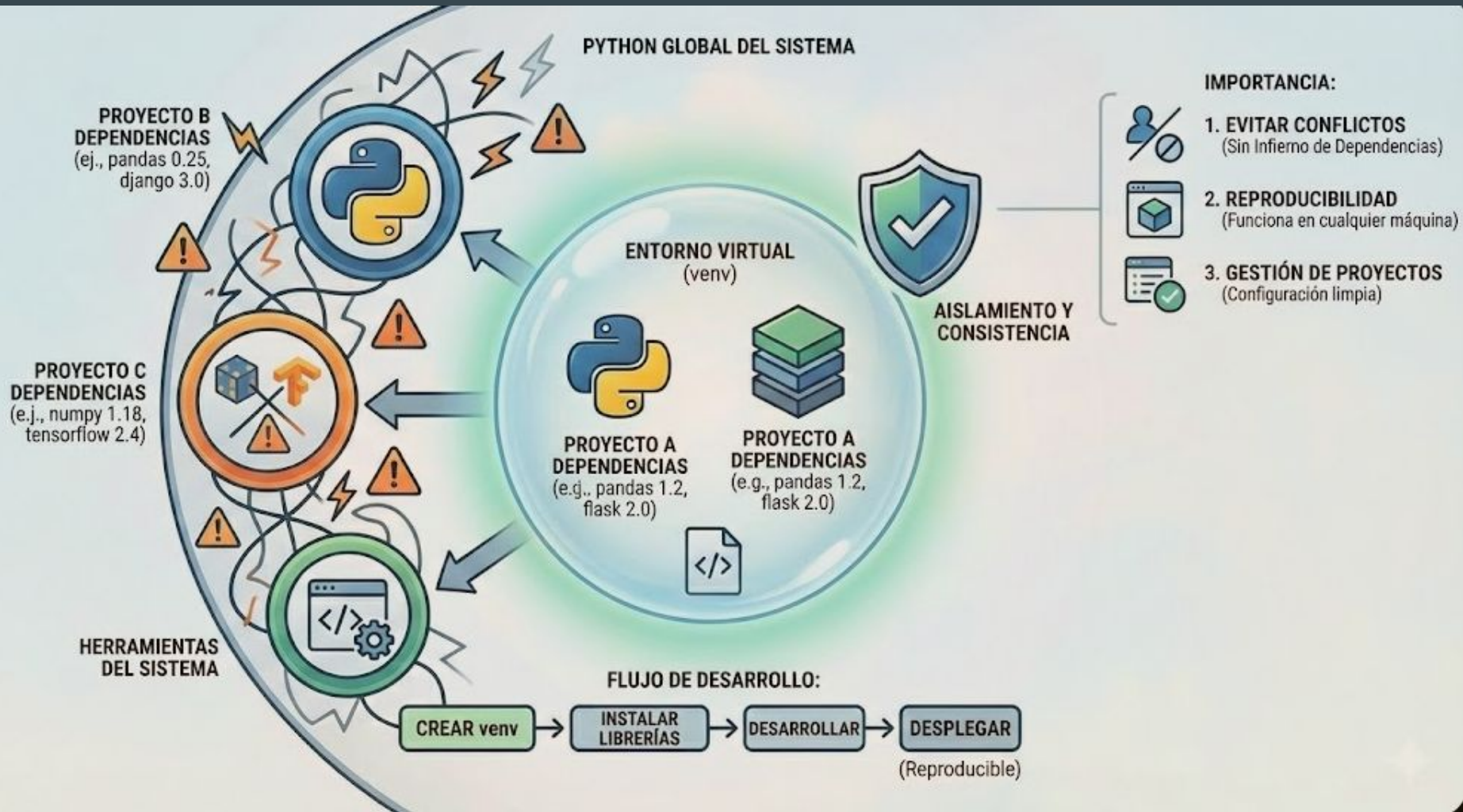
Temas

- Entorno Virtual
 - creación
 - activación
 - instalación de paquetes
 - buenas prácticas
- Nuevo proyecto Django
 - Estructura del proyecto
- Comandos Básicos
- Creación de Aplicación de Django
 - conectar la app con el proyecto
- Hola Mundo
- Manejo de peticiones (URLs)

Entorno Virtual

Un *entorno virtual* es un directorio que contiene una instalación de Python de una versión particular, además de una serie de paquetes adicionales.

Permite aislar las dependencias de diferentes proyectos, evitando **conflictos** entre las versiones de los paquetes.



Creación de Entorno Virtual

1. Una vez dentro de nuestra carpeta de trabajo...
2. Creamos el entorno virtual de python
 - a. `python -m venv venv` #por convención el nombre es “venv” pero puede ser cualquier cosa
3. Activando entorno virtual
 - a. `source venv/bin/activate` #en linux y mac
 - b. `source venv/bin/Activate.ps1`
4. Instalamos paquetes
 - a. `pip install django`
 - b. `pip list` #listamos los paquetes instalados
 - c. `pip freeze` #listamos los paquetes instalados en formato dependencia
 - d. `pip freeze > requirements.txt`
5. Buenas Prácticas
 - a. siempre usar la versión exacta: ej: `django==5.2.10`

Nuevo Proyecto

1. Después de instalar django

- a. `django-admin startproject src`
 - i. `# startproject` → Genera los archivos necesarios para iniciar un proyecto django
 - ii. `src` → nombre del proyecto

Analicemos la estructura del proyecto:

```
.../Ejemplos-Codigo/creacion-venv main ● ? × tree -l src
```

```
src1  
├── manage.py5  
└── src2  
    ├── asgi.py  
    ├── __init__.py  
    ├── settings.py3  
    ├── urls.py4  
    └── wsgi.py
```

1. Proyecto
2. Carpeta de configuración del proyecto
3. Archivo de configuración del proyecto, idioma, zona horaria, DB, etc.
4. Manejador de rutas principal
5. App principal del proyecto: inicia el servidor, crea base de datos, etc.

Comandos básicos

- python manage.py ...→ con este archivo se ejecuta todo lo referente al proyecto
- ./manage.py runserver → corre el servidor con parametros por default
- ./manage.py runserver 0.0.0.0:8000 → se puede especificar una ip y puerto
- ./manage.py migrate → crea la base de datos con la estructura necesaria de django, se ejecuta al crear el proyecto y después de realizar un makemigrations.
- ./manage.py makemigrations verifica si existen cambios en los modelos y genera los archivos ejecutables listos para migrar a la base de datos
- ./manage.py showmigrations → muestra las migraciones realizadas
- ./manage.py createsuperuser → crea el super usuario para acceder al panel de administración
- ./manage.py startapp nombre_app → crea una aplicacion dentro del proyecto

Creando una aplicación dentro del proyecto

```
python manage.py startapp MiApp
```

```

.../creacion-venv/src main ● ? } ls
Permissions Size User      Date Modified Name
drwxr-xr-x   - juanluis  2 Feb 20:30  src
.rwxr-xr-x  659 juanluis  2 Feb 16:08  manage.py

.../creacion-venv/src main ● ? ① python manage.py startapp MiApp

.../creacion-venv/src main ● ? } ls
Permissions Size User      Date Modified Name
drwxr-xr-x   - juanluis  2 Feb 20:31  MiApp ②
drwxr-xr-x   - juanluis  2 Feb 20:30  src
.rwxr-xr-x  659 juanluis  2 Feb 16:08  manage.py

.../creacion-venv/src main ● ? } tree MiApp
MiApp
├── admin.py ⑤
├── apps.py
├── __init__.py
├── migrations
│   └── __init__.py
├── models.py ③
├── tests.py
└── views.py ④

2 directories, 7 files

```

1. Se crea la App
2. Se crea carpeta con el nombre
3. Archivo para Modelos
4. Views de la app
5. Archivo para conectar la app al panel de administración

Conectar la App con el proyecto

Después de crear la App debemos informarle a django que forma parte del proyecto, de esta manera cuando busquemos cambios en modelos nos apliquen los cambios en la base de datos, así como todas las prestaciones que ofrece el framework con las aplicaciones.

src

- src
 - MiApp
 - migrations
 - __init__.py
 - admin.py
 - apps.py
 - models.py
 - tests.py
 - views.py
- src
 - __pycache__
 - __init__.py
 - asgi.py
 - settings.py
 - urls.py
 - wsgi.py
 - manage.py
- venv
- requirements.txt

```
28 ALLOWED_HOSTS = []
29
30
31 # Application definition
32
33 INSTALLED_APPS = [
34     'django.contrib.admin',
35     'django.contrib.auth',
36     'django.contrib.contenttypes',
37     'django.contrib.sessions',
38     'django.contrib.messages',
39     'django.contrib.staticfiles',
40     'MiApp.apps.MiappConfig'
41 ]
42
43 MIDDLEWARE = [
44     'django.middleware.security.SecurityMiddleware',
45     'django.contrib.sessions.middleware.SessionMiddleware',
46     'django.middleware.common.CommonMiddleware',
47     'django.middleware.csrf.CsrfViewMiddleware',
48     'django.contrib.auth.middleware.AuthenticationMiddleware',
49     'django.contrib.messages.middleware.MessageMiddleware',
50     'django.middleware.clickjacking.XFrameOptionsMiddleware',
```

Corriendo el servidor por primera vez

```
.../creacion-venv/src main • ? > python manage.py runserver
```

```
Watching for file changes with StatReloader
```

```
Performing system checks...
```

```
System check identified no issues (0 silenced).
```

```
You have 18 unapplied migration(s). Your project may not work properly until you apply the migrations for app(s):  
admin, auth, contenttypes, sessions.
```

```
Run 'python manage.py migrate' to apply them.
```

```
February 03, 2026 - 04:15:46
```

```
Django version 6.0.1, using settings 'src.settings'
```

```
Starting development server at http://127.0.0.1:8000/
```

```
Quit the server with CONTROL-C.
```

```
WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI server instead.
```

```
For more information on production servers see: https://docs.djangoproject.com/en/6.0/howto/deployment/
```

```
[03/Feb/2026 04:16:06] "GET / HTTP/1.1" 200 12068
```

```
Not Found: /favicon.ico
```

```
[03/Feb/2026 04:16:06] "GET /favicon.ico HTTP/1.1" 404 2205
```



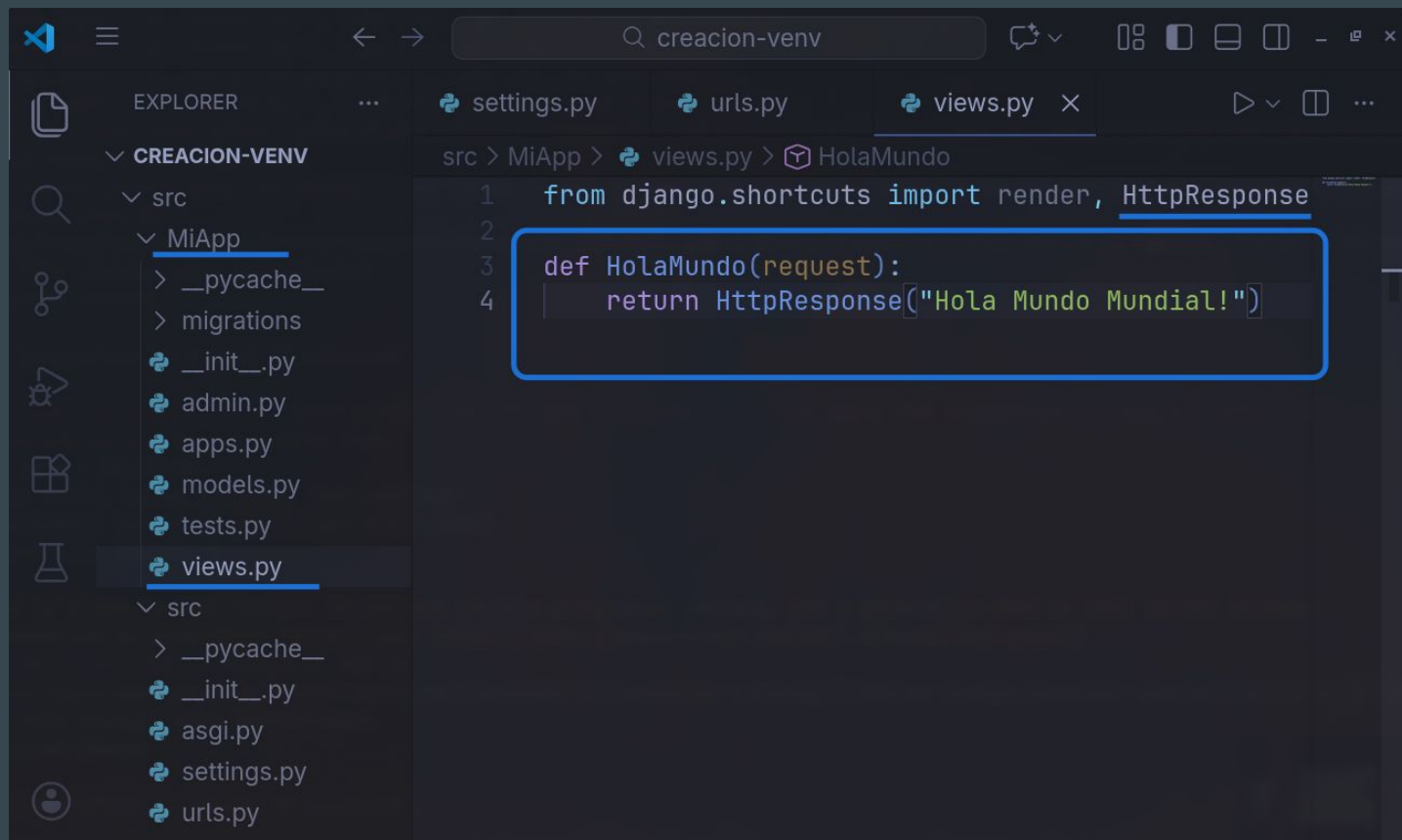
The install worked successfully!
Congratulations!

View [release notes](#) for Django 6.0

You are seeing this page because `DEBUG=True` is in
your settings file and you have not configured any
URLs.

django

Hola Mundo | views.py



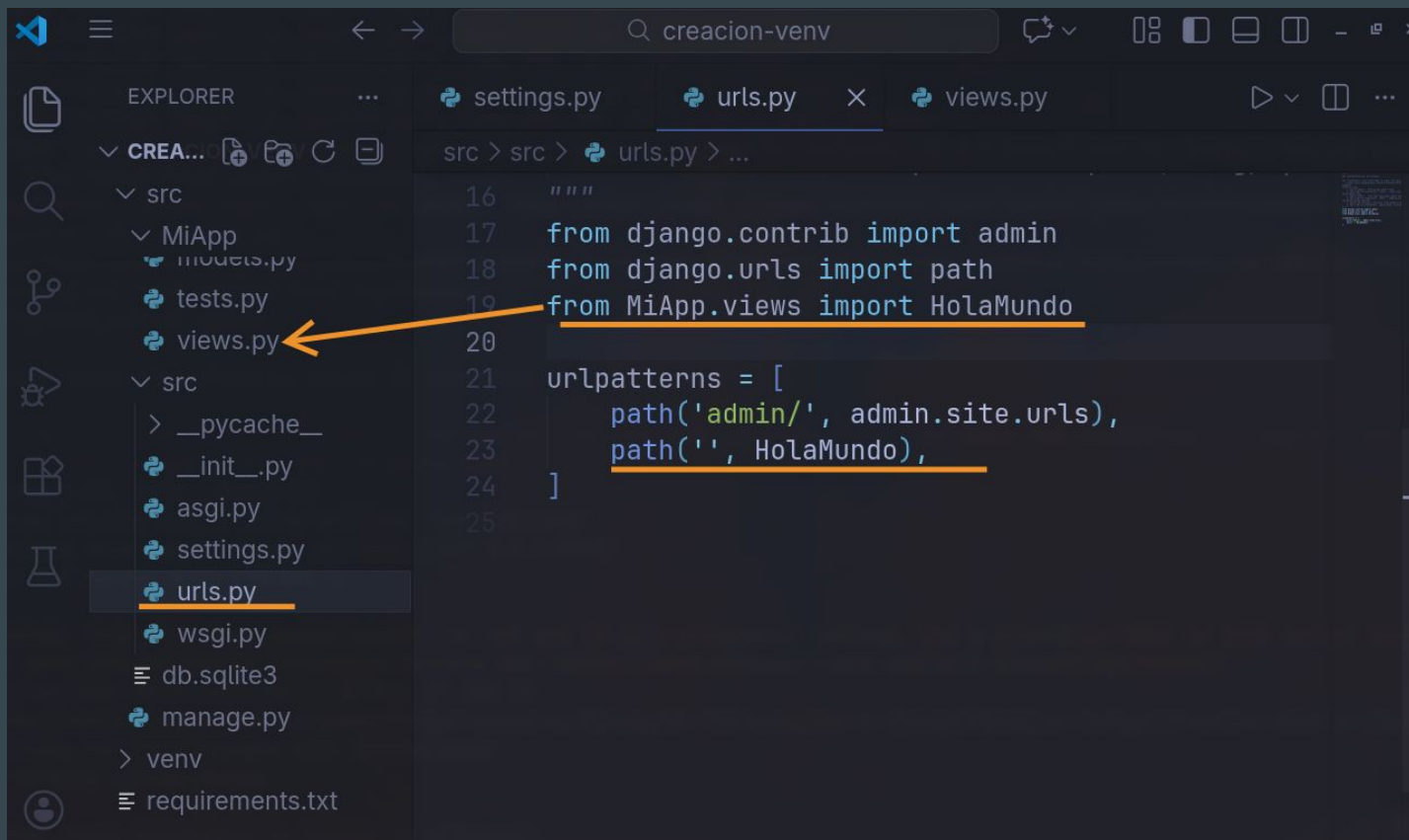
The image shows a code editor window with the following structure:

- EXPLORER**
 - CREACION-VENV
 - src
 - MiApp
 - __pycache__
 - migrations
 - __init__.py
 - admin.py
 - apps.py
 - models.py
 - tests.py
 - views.py**

- FILE EXPLORER**
- src > MiApp > views.py > HolaMundo
- EDITOR**

```
1 from django.shortcuts import render, HttpResponse
2
3 def HolaMundo(request):
4     return HttpResponse("Hola Mundo Mundial!")
```


Hola Mundo | urls.py



The image shows a Visual Studio Code editor window with a Django project. The Explorer sidebar on the left shows the project structure: `src` > `MiApp` > `views.py`. The `urls.py` file is selected and highlighted with an orange underline. An orange arrow points from the `views.py` file in the Explorer to the `from MiApp.views import HolaMundo` line in the code editor. The code editor shows the following Python code:

```
16 """
17 from django.contrib import admin
18 from django.urls import path
19 from MiApp.views import HolaMundo
20
21 urlpatterns = [
22     path('admin/', admin.site.urls),
23     path('', HolaMundo),
24 ]
25
```



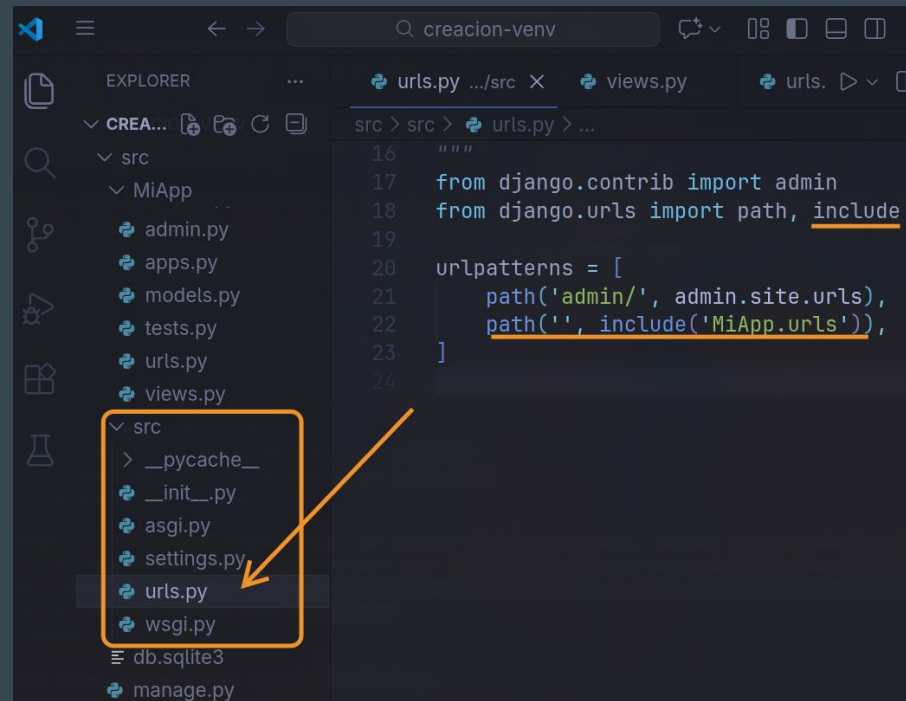
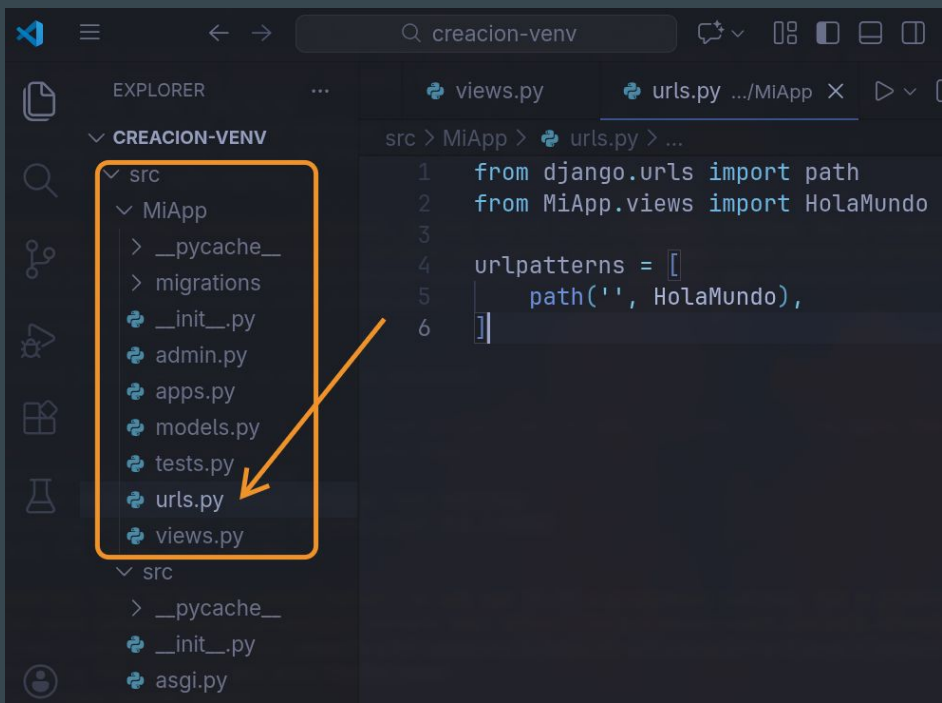

http://localhost:8000

Hola Mundo Mundial!

Buenas prácticas

Separar el manejo de rutas/peticiones del proyecto es bueno llevarlo a cabo desde el principio, MiApp es nuestra app principal debemos ceder el manejo principal de las urls.

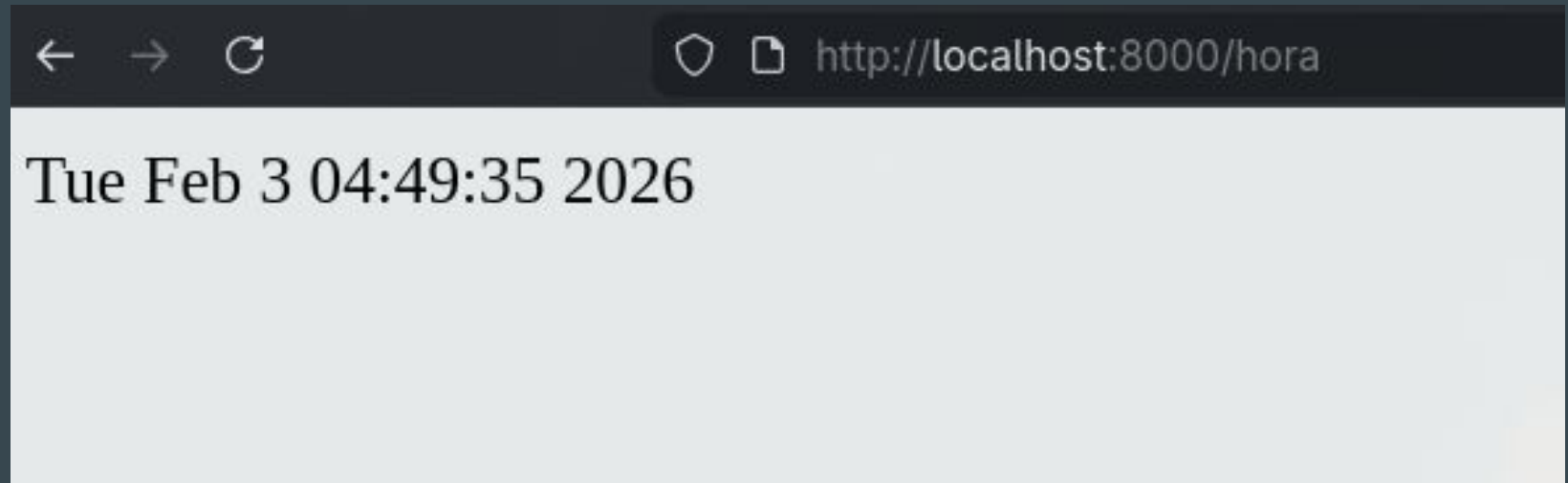
1. Crear dentro de MiApp un archivo [urls.py](#)
2. crear las rutas necesarias, en este caso migrar la de HolaMundo



Actividad

en clase

Crear una nueva ruta, donde imprima la fecha y la hora del sistema.



Resultado deseado.